FLEXIBLE JOB SHOP SCHEDULING PROBLEM WITH TIME WINDOWS

Siwen Liu
School of Information Science and Technology
Sandau University
Shanghai
siwenliu@sandau.edu.cn

August 10, 2025

ABSTRACT

1 Introduction

Community group buying has rapidly gained popularity as an e-commerce model, offering consumers a convenient way to purchase fresh products through online platforms, with the option to pick up items at designated locations the following day. This model streamlines the supply chain by reducing intermediaries, which lowers product prices and improves cost-effectiveness. It has become especially popular among urban consumers in China, addressing the demand for both convenience and affordability. The supply chain in community group buying consists of three main components: warehouses, distributors, and self-pickup points. Warehouses process and sort goods from suppliers according to distributor needs, and ship them to distributors on time. Distributors then perform secondary sorting before delivering goods to self-pickup points for consumer collection. Efficient warehouse operations are critical to meeting tight delivery deadlines and minimizing operational costs. Despite the advantages, the scheduling of warehouse operations presents several challenges. Goods in the warehouse must undergo different processing steps (e.g., shelving, processing, sorting), and resources such as sorting docks and processing lines are limited. Moreover, suppliers deliver goods in multiple batches based on predicted demand, leading to dynamic and unpredictable arrivals. Each distributor also has personalized delivery requirements based on order volume, product type, and the distance to self-pickup points. These complexities make it difficult to optimize warehouse schedules while minimizing production costs and ensuring timely deliveries.

Thus, this research focuses on optimizing warehouse scheduling in community group buying, addressing these challenges by modeling the problem as a flexible job shop scheduling problem (FJSP). The aim is to develop a solution that minimizes cost and maximizes efficiency, considering dynamic deliveries, limited processing resources, and varying delivery requirements.

1.1 Motivation

The rapid emergence of *New Retail*—an innovative commerce model that integrates online platforms with offline fulfillment infrastructures—has significantly transformed traditional supply chains into highly responsive, data-driven ecosystems. In this paradigm, customer demands are collected in real time through online channels and immediately propagated upstream to manufacturing and distribution units. Consequently, production systems are increasingly required to operate under dynamic and uncertain conditions, where job requests do not follow a fixed schedule but instead arrive sporadically and unpredictably in response to volatile consumer behavior.

This shift presents profound challenges to conventional production scheduling methods, which are typically designed for static environments with complete information known in advance. In particular, the emergence of **dynamic job arrivals** necessitates scheduling models that are capable of real-time responsiveness. In community group-buying and on-demand fulfillment platforms—key application scenarios under New Retail—consumer orders are often consolidated

within short time windows and subsequently transmitted to central warehouses or manufacturing centers for rapid execution. These production jobs are then processed and dispatched to geographically distributed distributors, all under tight time constraints and strict delivery commitments.

To meet these requirements, central warehouse operations must simultaneously address two tightly coupled decision-making stages: (i) the scheduling of incoming production jobs on a set of flexible machines, and (ii) the batching and dispatching of completed jobs to external distributors. The complexity of this *Integrated Flexible Job Shop Scheduling and Dispatching Problem with Dynamic Job Arrivals* (IFJSSP-DP-DJA) stems not only from the combinatorial nature of each subproblem, but also from the need to integrate them under real-time, incomplete, and evolving information flows.

Traditional optimization-based scheduling techniques often struggle to provide timely solutions in such dynamic settings due to their computational complexity and lack of adaptability. In contrast, advanced learning-based methods—particularly those grounded in **deep reinforcement learning (DRL)**—offer promising capabilities for reactive decision-making, scalability, and policy generalization in uncertain environments. This motivates the development of novel scheduling frameworks that can continuously learn and adapt under the evolving conditions of New Retail-driven production-distribution systems.

2 Literature Review

3 Problem Formulation

This study addresses the Integrated Flexible Job Shop Scheduling and Dispatching Problem under Dynamic Job Demand (IFJSSP-DP-DJD), which arises in production environments where a set of jobs arrives dynamically over time. These jobs must be processed and subsequently dispatched to multiple distributors, subject to specific delivery requirements.

The concept of dynamic job demand refers to a scenario where job requests are driven by real-time consumer orders, resulting in unpredictable and time-varying job arrivals. Unlike traditional batch-oriented production systems, where job arrivals are generally predetermined, dynamic job demand introduces significant uncertainty as jobs are continuously generated by customer demands, often with varying processing times and delivery constraints. This situation is particularly prevalent in the context of modern retail business models, where online demand is tightly integrated with offline fulfillment systems. Consumer orders are transmitted in near real-time to upstream production systems, creating a dynamic flow of job requests that are unpredictable throughout the scheduling horizon.

The central challenge of this problem lies in the simultaneous optimization of two interdependent decision stages:

Job Scheduling in a Flexible Job Shop: In the processing stage, each job consists of a predefined sequence of operations. These operations can be processed on multiple alternative machines, with machine-dependent and deterministic processing times. However, the exact arrival times of jobs are not known in advance and are subject to dynamic fluctuations. This necessitates a responsive and adaptive scheduling strategy that can make real-time decisions as new jobs arrive. Key decisions include assigning operations to machines, sequencing operations while adhering to precedence constraints, and ensuring machine exclusivity and capacity limitations.

Job Dispatching and Batching: Upon completion of processing, jobs must be grouped into batches and dispatched to designated distributors. Each batch incurs a fixed base loading time and an additional variable time proportional to its size. A batch can only be dispatched once all jobs within it are fully processed and loaded. Furthermore, each job is uniquely assigned to one batch for its corresponding distributor. Distributors have specific delivery requirements, including a required completion percentage of jobs and a due date by which the batch must be dispatched. Tardiness in meeting these delivery requirements incurs a penalty, with the penalty weight reflecting the importance of each requirement.

The objective of this study is to develop an integrated scheduling and dispatching plan that minimizes the total weighted tardiness of all delivery requirements. The dynamic nature of job demand—driven by the variability in end-user demand and the real-time requirements of order fulfillment—introduces temporal uncertainty into the system. This uncertainty necessitates the development of a flexible, event-driven scheduling framework capable of incrementally updating schedules in real time to ensure timely job fulfillment and efficient resource utilization.

3.1 Mathematical Model

To address the dynamic job arrival characteristics of the Integrated Flexible Job Shop Scheduling and Dispatching Problem (IFJSSP-DP), we formulate a Mixed Integer Linear Programming (MILP) model that is solved repeatedly in a

rolling horizon fashion. At each decision point t, only the set of jobs that have arrived up to time t are included in the model.

This subsection summarizes the notation used in the proposed MILP model for the Integrated Flexible Job Shop Scheduling and Dispatching Problem (IFJSSP-DP). Let J_t denote the set of dynamically released jobs at the current decision epoch. Each job $j \in J_t$ consists of a sequence of operations O_j, where each operation must be assigned to one eligible machine $m \in K_{jo}$. The model integrates production scheduling and batch-based dispatching under multiple delivery requirements R, considering job arrival times, precedence constraints, machine assignment, delivery deadlines, and partial fulfillment targets.

Table 1: Model Parameters and Decision Variables

Symbol	Description
Indices and Sets	
J_t	Set of all jobs released up to time t
O_j	Set of operations for job j
K_{jo}	Set of eligible machines for operation o of job j
$\mathcal{B}$	Set of dispatch batches
R	Set of delivery requirements
Parameters	
a_j	Arrival time of job j
δ_j	Due date of job j
p_{jom}	Processing time of operation o of job j on machine m
B	A large constant for disjunctive constraints
q_{jr}	Quantity of job j contributing to delivery requirement r
Q_r	Total demand required by delivery requirement r
ρ_r	Minimum fulfillment ratio for requirement r
w_r	Penalty weight for delivery shortfall in requirement r
Decision Variables	
x_{jom}	Binary; 1 if operation o of job j is assigned to machine m
s_{jo}	Start time of operation o of job j
C_j	Completion time of job j
T_j	Tardiness of job j
y_{jb}	Binary; 1 if job j is assigned to batch b
d_b	Dispatch time of batch b
z_{br}	Binary; 1 if batch b serves delivery requirement r
w_{jbr}	Binary; 1 if job j in batch b contributes to requirement r
y_{jokl}	Binary; 1 if operation (j, o) precedes (k, l) on the same machine
u_r	Delivery shortfall quantity for requirement r

The formulation is as follows:

$$\min \quad \sum_{j \in J_t} T_j + \sum_{r \in R} w_r \cdot u_r \tag{1}$$

$$\text{s.t.} \quad \sum_{m \in K_{jo}} x_{jom} = 1 \qquad \forall j \in J_t,\ \forall o \in O_j \quad \text{(operation assignment)} \tag{2}$$

$$s_{j,o+1} \geq s_{jo} + \sum_{m \in K_{jo}} p_{jom} x_{jom} \qquad \forall j \in J_t,\ o < |O_j| \quad \text{(precedence)} \tag{3}$$

$$s_{jo} \geq a_j \qquad \forall j \in J_t,\ \forall o \in O_j \quad \text{(arrival time)} \tag{4}$$

$$s_{jo} \geq s_{kl} + \sum_{m \in M} p_{klm} x_{klm} - B(1 - y_{jokl}) \qquad \forall (jo), (kl) \text{ on same } m \quad \text{(disjunctive 1)} \tag{5}$$

$$s_{kl} \geq s_{jo} + \sum_{m \in M} p_{jom} x_{jom} - B y_{jokl} \qquad \forall (jo), (kl) \text{ on same } m \quad \text{(disjunctive 2)} \tag{6}$$

$$C_j \geq s_{j,o_j} + \sum_{m \in K_{j,o_j}} p_{j,o_j,m} x_{j,o_j,m} \qquad \forall j \in J_t \quad \text{(completion time)} \tag{7}$$

$$T_j \geq C_j - \delta_j \qquad \forall j \in J_t \quad \text{(tardiness)} \tag{8}$$

$$T_j \geq 0, \quad s_{jo} \geq 0, \quad C_j \geq 0 \qquad \forall j,\ o \in O_j \quad \text{(non-negativity)} \tag{9}$$

$$\sum_{b \in \mathcal{B}} y_{jb} = 1 \qquad \forall j \in J_t \quad \text{(job assigned to one batch)} \tag{10}$$

$$d_b \geq C_j - M(1 - y_{jb}) \qquad \forall j \in J_t,\ \forall b \in \mathcal{B} \quad \text{(dispatch after completion)} \tag{11}$$

$$w_{jbr} \leq y_{jb} \qquad \forall j \in J_t,\ b \in \mathcal{B},\ r \in R \tag{12}$$

$$w_{jbr} \leq z_{br} \qquad \forall j \in J_t,\ b \in \mathcal{B},\ r \in R \tag{13}$$

$$w_{jbr} \geq y_{jb} + z_{br} - 1 \qquad \forall j \in J_t,\ b \in \mathcal{B},\ r \in R \tag{14}$$

$$\sum_{j \in J_t} \sum_{b \in \mathcal{B}} q_{jr} \cdot w_{jbr} \geq \rho_r Q_r - u_r \qquad \forall r \in R \quad \text{(delivery coverage)} \tag{15}$$

$$z_{br} \geq w_{jbr} \qquad \forall j \in J_t,\ b \in \mathcal{B},\ r \in R \quad \text{(batch activates delivery)} \tag{16}$$

The proposed MILP model aims to minimize the total job tardiness and delivery shortfall penalty in an integrated flexible job shop scheduling and batch dispatching environment with dynamic job arrivals. Constraint 1 ensures that each operation of every job is assigned to exactly one eligible machine, thereby respecting machine eligibility constraints. Constraint 2 preserves the technological precedence among operations within each job by enforcing that a subsequent operation starts only after the preceding one finishes. Constraint 3 incorporates the dynamic nature of job arrivals by restricting operation start times to be no earlier than the job's arrival time. Constraints 4 and 5 constitute the disjunctive constraints that prevent time overlaps between operations assigned to the same machine, using a binary sequencing variable to determine the execution order of operation pairs. Constraint 6 computes the completion time of each job as the end time of its last operation, taking into account the selected machine and processing time. Constraint 7 models job-level tardiness as the positive deviation of completion time from the job's due date. Constraint 8 guarantees the non-negativity of all time-related decision variables. Constraint 9 requires that each job be assigned to exactly one dispatch batch, enabling its inclusion in a feasible delivery plan. Constraint 10 ensures that any dispatch batch is scheduled only after the completion of all jobs it contains. Constraints 11 through 13 define the binary indicator for whether a job assigned to a batch contributes to a particular delivery requirement, using standard linearization techniques to model logical conjunctions. Finally, Constraint 14 ensures that each delivery requirement is fulfilled at or above its specified threshold level, while allowing controlled violation through a slack variable that is penalized in the objective function. Together, the constraints collectively define a tightly integrated scheduling and dispatching optimization problem that captures job-shop flexibility, delivery deadlines, and batch-level coordination in a dynamic production environment.

To handle dynamic arrivals, the model is executed periodically in a rolling horizon fashion. At each decision epoch, only the subset of jobs that have already arrived ($a_j \leq t$) are included in the optimization. This allows the model to remain computationally tractable while maintaining responsiveness to real-time job inflow.

This formulation enables real-time responsiveness to job arrivals and system state changes, and serves as a near-optimal baseline for evaluating learning-based and heuristic scheduling methods.

3.2 Theoretical Support for Scheduling Policy Design

In this section, we present theoretical results that support the design of reward functions and prioritization strategies in the proposed HDRL scheduling framework. These lemmas are particularly relevant for Chapter 3, where the Integrated Flexible Job Shop Scheduling and Dispatching Problem (IFJSSP-DP) is formulated. The following results provide insights into task sequencing and batch dispatch strategies that help accelerate learning and improve system performance.

Additionally, we consider an extension to the original problem formulation by incorporating uncertainty in job processing times. Specifically, we assume that the actual processing time of each operation is a random variable whose value is not known until the operation begins. This extension reflects practical scenarios such as human-performed tasks, unpredictable delays in machinery, or variability in product handling, and introduces an additional layer of complexity to the scheduling environment.

In the updated problem setting, let the processing time $P_{j,o,m}$ of operation o for job j on machine m be a random variable, denoted as $\tilde{P}_{j,o,m}$. The realization of $\tilde{P}_{j,o,m}$ is unknown until the operation is initiated. Consequently, the scheduler must make dispatching and sequencing decisions based on historical estimates, expected values $\mathbb{E}[\tilde{P}_{j,o,m}]$, or sampling-based predictions, rather than deterministic values.

This uncertainty affects both stages of the IFJSSP-DP:

- In the processing stage, machine allocation and sequencing must be robust to variability in execution times, potentially relying on adaptive policies that update as real-time feedback becomes available.
- In the dispatching stage, the uncertainty in completion times propagates to uncertainty in dispatch timing, affecting batch formation and due-date compliance. The reward design in HDRL must therefore penalize both high variance and systematic delay.

The following theoretical results still apply in expectation, and guide how the HDRL framework can prioritize low-risk (low-variance) actions when optimizing under uncertainty.

Lemma 1. *Let $\mathcal{J}' = \{J_1, J_2, \ldots, J_n\}$ be a set of n jobs ready for processing on a single shared machine, with random processing times $\tilde{p}_j$ having known expectations $\mu_j = \mathbb{E}[\tilde{p}_j]$. Let w_j be the weight associated with job J_j. Then, scheduling the jobs in ascending order of μ_j minimizes the expected total weighted waiting time:*

$$\mathbb{E}\left[\textit{Total Waiting Time}\right] = \sum_{j=1}^{n} w_j \cdot \mathbb{E}[W_j]$$

where W_j is the waiting time experienced by job J_j.

Proof. Let π be a job sequence and $W_j^{(\pi)}$ the waiting time for job J_j under π. Since waiting times depend linearly on the sum of preceding job durations, we have:

$$\mathbb{E}[W_j^{(\pi)}] = \sum_{k=1}^{j-1} \mu_{\pi(k)}$$

Thus the total expected weighted waiting time is:

$$\mathbb{E}\left[\sum_{j=1}^{n} w_j W_j^{(\pi)}\right] = \sum_{j=1}^{n} w_j \sum_{k=1}^{j-1} \mu_{\pi(k)} = \sum_{1 \leq k < j \leq n} w_j \mu_{\pi(k)}$$

To minimize this sum, we arrange μ_j in ascending order. The proof parallels the deterministic SPT case, extended to expected values. □

Lemma 2. *Let batch b serve delivery requirement r with dispatch time t_b and due date δ_r. Let ρ_r be the required delivery percentage for requirement r, and w_r be the associated tardiness penalty weight. The penalty incurred for tardiness is:*

$$L_b = w_r \cdot \rho_r \cdot \max(0, t_b - \delta_r)$$

Then, L_b is monotonically increasing with ρ_r, assuming all other variables are fixed.

Proof. We analyze the derivative of L_b with respect to ρ_r. Define $\Delta = \max(0, t_b - \delta_r)$, which is non-negative by definition. Then:

$$\frac{\partial L_b}{\partial \rho_r} = w_r \cdot \Delta$$

Since $w_r > 0$ and $\Delta \geq 0$, we conclude that L_b is monotonically increasing with ρ_r. Hence, batches fulfilling denser delivery requirements incur higher potential penalty when delayed. This result justifies reward design strategies that prioritize jobs with high-density delivery constraints. □

Proposition 1. *In the low-level policy learning process of HDRL, incorporating priority-based subgoals (e.g., tagging urgent jobs or vehicles) leads to faster convergence of the policy network under sparse reward conditions.*

Proof. Let π_{std} be a low-level policy trained with standard reward, and π_{shaped} a policy trained with additional subgoal shaping signals for priority tasks. Under sparse reward settings, gradient signals $\nabla J(\theta)$ computed via temporal difference (TD) updates are weak, leading to slow updates in π_{std}.

When priority tasks are tagged and rewarded (shaped), we effectively increase the frequency of reward events, improving the temporal credit assignment. This results in stronger TD error signals, leading to larger and more informative gradient updates:

$$\nabla J(\theta) \propto \mathbb{E}[\delta_t \nabla_\theta \log \pi_\theta(a_t|s_t)]$$

With reward shaping, δ_t is larger in magnitude and more frequent, thus accelerating convergence. Empirical studies in curriculum learning and reward shaping reinforce this theoretical insight. □

These theoretical results validate the structure of the hierarchical reinforcement learning framework proposed in Chapter 4, and justify key design choices in reward shaping, scheduling prioritization, and dispatch coordination.

4 Solution Methodology

This section outlines the proposed hierarchical reinforcement learning (HRL) framework developed to solve the integrated scheduling and dispatching problem in flexible job shop environments. The solution architecture adopts a two-level decision-making structure, with a meta-level reinforcement learning agent orchestrating the overall control policy and two task-specific agents handling scheduling and dispatching at the operational level. Figure 2 illustrates the hierarchical framework comprising high-level coordination and low-level execution modules.

At the upper level, a meta-controller is implemented as a policy network trained via policy gradient methods. This agent receives a compact nine-dimensional state vector encoding global workshop status—including job urgency, machine load balance, job completion ratio, and scheduling recency—and selects one of three operational modes: (i) scheduling with a Rule-DQN agent, (ii) dispatching via a heuristic batching policy, or (iii) waiting when no feasible action is detected. The meta-agent incorporates an upper confidence bound (UCB) strategy to balance exploration and exploitation, and action masking to enforce operational feasibility.

At the lower level, the framework integrates two distinct action modules. The scheduling module is realized through a hybrid Rule-DQN agent, which combines the domain knowledge of classical dispatching rules with the adaptability of deep Q-learning. Instead of directly outputting scheduling actions, the agent learns to select from a portfolio of eight widely used rules (e.g., SPT, EDD, CR), enabling both interpretability and optimization. Meanwhile, the dispatching module relies on a scoring-based heuristic that prioritizes completed jobs for delivery based on a delay-normalized urgency metric. The batching process groups jobs into delivery sets based on a fixed time window and distributor constraints.

Importantly, unlike conventional bi-level optimization frameworks (e.g., Feng, Che, and Lei 2024; Zhao et al. 2023) or embedded optimization-learning systems (Yan et al. 2024), this framework does not treat the lower-level as a black-box solver invoked by the upper agent. Instead, it models the lower level as an adaptive environment whose response dynamics are learned by the meta-agent over time. This allows the system to flexibly accommodate diverse scheduling and dispatching behaviors, as the upper-level agent implicitly captures the capabilities of lower-level modules through episodic feedback and long-term reward shaping (cf. Su et al. 2023).

This design choice enhances the robustness and generalization of the overall scheduling framework, enabling it to adapt to varying production conditions and job characteristics without requiring explicit reconfiguration of lower-level policies.

4.1 Meta-Controller Design and Optimization

The meta-controller is formulated as a Markov Decision Process (MDP), denoted by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$. At each high-level decision step t, the agent observes a global state vector $s_t \in \mathbb{R}^9$, which encapsulates the overall production-dispatch environment dynamics. This state vector includes: the proportion of newly arrived jobs $q_{\text{new}} = N_{\text{new}}/N_{\text{available}}$, normalized machine load imbalance $\sigma_m = \text{StdDev}(\text{remaining\_time}_m)/\text{max\_processing\_time}$, urgency ratio $\rho_{\text{urgent}} = N_{\text{urgent}}/N_{\text{available}}$, normalized time since the last scheduling decision $\tau_{\text{sched}} = (t - t_{\text{last\_schedule}})/\text{max\_processing\_time}$, system pressure $\psi = (\sum_j p_j)/(\sum_m r_m)$, job completion and dispatch ratios $r_{\text{complete}} = N_{\text{completed}}/N_{\text{total}}$ and $r_{\text{dispatch}} = N_{\text{dispatched}}/N_{\text{total}}$, average machine utilization $\mu_{\text{util}} = \frac{1}{M}\sum_{m=1}^{M} \text{util}_m$, and average waiting time of available jobs $w_{\text{avg}} = \frac{1}{N_{\text{available}}}\sum_j w_j$.

The agent selects an action $a_t \in \mathcal{A} = \{\texttt{schedule}, \texttt{dispatch}, \texttt{wait}\}$ to determine the next control delegation. Due to dynamic arrivals and stochastic processing durations, the transition function $\mathcal{P}(s_{t+1} \mid s_t, a_t)$ is unknown and must be learned through experience. The reward function $\mathcal{R}$ is designed to provide structured feedback for learning an effective coordination policy.

For scheduling decisions, the reward is computed as:

$$r_t^{\text{schedule}} = \alpha_1 u_t + \alpha_2 p_t - \alpha_3 \sigma_t$$

where $u_t = \frac{1}{M}\sum_{m=1}^{M} \text{util}_m$ denotes the average machine utilization, $p_t = N_{\text{completed}}/N_{\text{total}}$ reflects the job processing progress, and $\sigma_t = \sqrt{\frac{1}{M}\sum_{m=1}^{M}(l_m - \bar{l})^2}$ measures load imbalance among machines. This reward encourages efficient resource usage while maintaining load balance.

For dispatching decisions, the reward includes both action-based and delivery-based components:

$$r_t^{\text{dispatch}} = \beta_1 \cdot \mathbb{I}_{\text{dispatch\_action}} + \beta_2 \cdot \mathbb{I}_{\text{dispatching}} + \beta_3 \cdot \mathbb{I}_{\text{dispatched}}$$

with binary indicators representing whether the dispatch action is selected, actively ongoing, or successfully completed. In addition, a tardiness penalty is imposed for late deliveries:

$$r_j^{\text{tardiness}} = -\max(0, d_j - \delta_j)$$

where d_j is the actual dispatch time and δ_j is the due time for job j. For each distributor requirement r, if the delivered quantity falls below a required threshold, an aggregated penalty is applied:

$$L_r = -w_r \cdot \max(0, \rho_r A_r - A_r^{\text{delivered}})$$

where ρ_r is the delivery ratio requirement, A_r the total demand, and w_r the penalty weight. All reward terms are clipped and normalized to improve training stability.

The policy network maps input states to action preferences using a three-layer feedforward architecture. The forward computation is defined as:

$$h_1 = \text{GeLU}(\text{LayerNorm}(W_1 s_t + b_1)) \tag{17}$$

$$h_1' = \text{Dropout}(h_1, p = 0.2) \tag{18}$$

$$h_2 = \text{GeLU}(W_2 h_1' + b_2) \tag{19}$$

$$\text{logits} = W_3 h_2 + b_3 \tag{20}$$

where $W_1 \in \mathbb{R}^{32\times 9}$, $W_2 \in \mathbb{R}^{32\times 32}$, $W_3 \in \mathbb{R}^{3\times 32}$ and b_1, b_2, b_3 are bias vectors. Layer normalization and dropout are used to stabilize training and reduce overfitting.

To ensure valid decision making, an action mask $m_t \in \{0, 1\}^3$ is applied:

$$\text{masked\_logits}_t = \text{logits}_t + \log m_t, \quad \log 0 = -\infty$$

To encourage exploration, an Upper Confidence Bound (UCB) term is added:

$$\text{UCB}_t(a) = \text{logits}_t(a) + \lambda \cdot \sqrt{\frac{\log T_t}{N_t(a) + \epsilon}}$$

where T_t is the total number of decisions taken, $N_t(a)$ is the count of action a, and ϵ is a small constant for numerical stability.

The action is selected from a softmax distribution over masked and UCB-adjusted logits:

$$\pi_\theta(a_t \mid s_t) = \frac{\exp(\mathrm{UCB}_t(a_t))}{\sum_{a \in \mathcal{A}} \exp(\mathrm{UCB}_t(a) \cdot m_t(a))}$$

The policy is trained with a policy gradient objective incorporating entropy regularization:

$$\mathcal{L}_{\text{meta}}(\theta) = -\mathbb{E}_{\pi_\theta}[\log \pi_\theta(a_t \mid s_t) \cdot R_t] - \beta \cdot \mathbb{H}[\pi_\theta(\cdot \mid s_t)]$$

where R_t is the cumulative discounted return and $\mathbb{H}[\cdot]$ denotes the Shannon entropy. Training is performed using the Adam optimizer with learning rate $\eta = 10^{-3}$, and gradient clipping $\|\nabla \mathcal{L}\| \leq 1.0$ is applied to ensure convergence.

Algorithm 1 Meta-Controller Decision and Update Process

```
Initialize meta-policy π_θ, action counters N(a), and total step counter T_max
for each episode e = 1, ..., E do
    Reset environment and observe global state s_0
    for each decision step t = 0, ..., T_max do
        Extract feature vector x_t from s_t
        Compute logits and apply action mask
        Apply UCB adjustment and sample action a_t ~ π_θ(a_t | x_t)
        Execute a_t, receive r_t and observe s_{t+1}
        Store (s_t, a_t, r_t, log π_θ(a_t), s_{t+1}) in buffer
        if update condition is met then
            Sample batch and compute L_meta
            Update parameters via policy gradient
        if termination condition is met then break
```

4.2 Low-Level Scheduling Agent: Rule-DQN Hybrid Architecture

The low-level scheduling agent employs a hybrid Rule-DQN architecture that combines the domain knowledge of classical scheduling rules with the adaptability of deep Q-learning. Rather than directly learning job-to-machine assignments, the agent learns to intelligently select from a portfolio of established dispatching rules based on the current system state, thereby leveraging both proven heuristics and data-driven optimization.

4.2.1 Rule-Guided DQN Scheduling

The Rule-DQN agent adopts a two-tier hybrid architecture that integrates deep reinforcement learning with rule-based scheduling. At the upper level, a Deep Q-Network (DQN) is utilized to select an appropriate dispatching rule according to the current status of the job shop environment. The decision is based on a 10-dimensional state vector $s_t \in \mathbb{R}^{10}$, which encapsulates core system characteristics. These include the number of available jobs $n_{\text{jobs}} = |\mathcal{J}_{\text{avail}}|$ and the number of jobs in the waiting queue $n_{\text{waiting}} = |\mathcal{J}_{\text{wait}}|$, as well as the average due date of all pending jobs $\bar{d} = \frac{1}{|\mathcal{J}_{\text{avail}}|} \sum_{j \in \mathcal{J}_{\text{avail}}} d_j$ and the average processing time $\bar{p} = \frac{1}{|\mathcal{J}_{\text{avail}}|} \sum_{j \in \mathcal{J}_{\text{avail}}} \bar{p}_j$. In addition, the machine utilization rate is given by $u_m = \frac{1}{M} \sum_{m=1}^{M} \mathbb{I}[m \text{ is busy}]$, and the current time is normalized by the maximum time horizon as $t_{\text{norm}} = \frac{t}{T_{\max}}$. The state also tracks the number of completed jobs n_{comp}, dispatched jobs n_{disp}, and the normalized durations since the last scheduling and dispatching decisions, defined respectively as $t_{\text{sched}} = \frac{t - t_{\text{last\_sched}}}{T_{\max}}$ and $t_{\text{batch}} = \frac{t - t_{\text{last\_batch}}}{T_{\max}}$. Additionally, it tracks the number of completed and dispatched jobs, along with the time elapsed since the last scheduling and batching actions. This compact representation enables the DQN to approximate the action-value function $Q(s_t, a; \theta)$, where each action corresponds to a specific dispatching rule.

Once a rule is selected, it is executed in the lower decision tier, where the corresponding heuristic logic is applied to construct job-to-machine assignments. The implemented rule set includes classical heuristics such as Earliest Due Date (EDD), Shortest Processing Time (SPT), Longest Processing Time (LPT), Critical Ratio (CR), First Come First Served (FCFS), Most Work Remaining (MWKR), Least Work Remaining (LWKR), and a randomized strategy (RAND) that encourages exploratory behavior. Among these, the Critical Ratio rule assigns dispatch priority to each job j based on the urgency-to-load ratio, calculated as follows:

$$\mathrm{CR}_j = \frac{d_j - t_{\text{current}}}{\sum_{k=c_j}^{|O_j|} \min_{m \in M_{jk}} p_{jkm}},$$

where d_j denotes the due date of job j, t_{current} is the current system time, c_j is the index of the current operation, and p_{jkm} represents the processing time of operation k on machine m. This hierarchical structure allows the agent to exploit domain knowledge through heuristic rules while adapting dynamically to the system's evolving state via deep learning-based rule selection.

To approximate the action-value function for rule selection, we employ a fully connected feedforward neural network with three layers. The network takes as input a 10-dimensional state vector $s_t \in \mathbb{R}^{10}$, which captures the current status of the job shop environment. The forward computation proceeds as follows: the first hidden layer computes $h_1 = \text{ReLU}(W_1 s_t + b_1)$, followed by a second hidden layer $h_2 = \text{ReLU}(W_2 h_1 + b_2)$. The final output layer yields the Q-values for all available dispatching rules via $Q(s_t, \cdot; \theta) = W_3 h_2 + b_3$, where the symbol "$\cdot$" indicates a vector of Q-values over the action space. Here, $W_1 \in \mathbb{R}^{128\times 10}$, $W_2 \in \mathbb{R}^{128\times 128}$, and $W_3 \in \mathbb{R}^{8\times 128}$ denote the weight matrices of the respective layers, and b_1, b_2, b_3 are the corresponding bias terms. The network employs ReLU as the activation function and a hidden dimension of 128 to provide sufficient representational capacity while maintaining computational efficiency. This architecture enables the agent to learn a mapping from system states to expected rule performance, supporting adaptive dispatching decisions under dynamic operating conditions.

4.2.2 Training Strategy and Job Assignment

The training procedure follows the standard Deep Q-Network (DQN) framework, with several enhancements for learning stability and effectiveness. An ϵ-greedy exploration strategy is adopted, where the exploration probability decays exponentially with time:

$$\epsilon_t = \epsilon_{\text{end}} + (\epsilon_{\text{start}} - \epsilon_{\text{end}}) \cdot \exp\left(-\frac{t}{\tau_{\text{decay}}}\right),$$

with $\epsilon_{\text{start}} = 0.3$, $\epsilon_{\text{end}} = 0.01$, and $\tau_{\text{decay}} = 10{,}000$. This formulation encourages extensive exploration in early training stages while gradually shifting toward policy exploitation.

To stabilize training, a target network $Q(s, a; \theta^-)$ is used, where parameters θ^- are periodically updated from the main network via:

$$\theta^- \leftarrow \theta,$$

with synchronization performed every 100 steps. This technique mitigates instability caused by non-stationary learning targets.

The network is trained by minimizing the temporal-difference loss using the mean squared error:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s',d)\sim\mathcal{D}}\left[\left(Q(s,a;\theta) - \left(r + \gamma \cdot \max_{a'} Q(s', a'; \theta^-)(1-d)\right)\right)^2\right],$$

where $\mathcal{D}$ denotes the experience replay buffer, $\gamma = 0.99$ is the discount factor, and d indicates whether the episode has terminated. A replay buffer of capacity 10,000 stores tuples $(s_t, a_t, r_t, s_{t+1}, d_t)$ and enables mini-batch training by breaking the temporal correlation among samples.

After the rule is selected by the Q-network, job-to-machine assignment is performed using a greedy earliest-available strategy. For each prioritized job j, the agent selects the machine m_j^* from its eligible set M_j according to:

$$m_j^* = \arg\min_{m\in M_j} \begin{cases} 0 & \text{if machine } m \text{ is idle,} \\ t^m_{\text{remaining}} & \text{otherwise,} \end{cases}$$

where $t^m_{\text{remaining}}$ represents the remaining processing time on machine m. Once a machine is assigned to a job, it is excluded from further consideration within the current decision epoch to avoid conflict and maintain one-to-one assignments. This ensures feasible, conflict-free scheduling within each time step.

To prevent multiple jobs from being assigned to the same machine within a single scheduling decision, the agent maintains a set of assigned machines and excludes them from subsequent allocations during the same time step.

This hybrid architecture enables the agent to benefit from established scheduling knowledge while adapting rule selection to specific operational contexts, combining interpretability with learning-based optimization in dynamic job shop environments.

4.3 Heuristic-Based Dispatching Agent for Batch Coordination

The dispatching agent serves as a rule-based module responsible for grouping completed jobs into delivery batches and coordinating their dispatch to downstream distributors. Unlike the upper-layer scheduling policy that leverages deep reinforcement learning, this component is designed to be lightweight, interpretable, and tightly aligned with established logistics heuristics.

To balance efficiency and freshness in delivery, the agent adopts a time-window-based batching policy. Completed jobs are first segregated by distributor ID to ensure that each batch corresponds to a single delivery destination. Within each distributor group, jobs are sorted by their completion timestamps and clustered into discrete time windows of fixed duration Δt, typically set to 8 time units. This rolling-window strategy ensures that jobs completed within similar time frames are dispatched together, reducing delivery latency without incurring excessive batch fragmentation. The system allows a maximum of W active time windows (default $W = 2$), maintaining a trade-off between batch granularity and dispatch frequency.

Priority within each batch is determined by a composite heuristic score:

$$\text{Score}(j, t) = -\frac{w_j(t)}{p_j^2}, \quad w_j(t) = \max(0, t - t_j^{\text{comp}})$$

where $w_j(t)$ is the time elapsed since job j completed and p_j denotes its processing time. The quadratic penalty in the denominator discourages computationally intensive jobs from delaying simpler ones, while the negative sign ensures that longer-waiting jobs are prioritized. This mechanism promotes responsiveness while mitigating starvation.

The dispatching process is governed by a structured algorithm, summarized below:

Algorithm 2 Heuristic-Based Batch Assignment

Initialize available windows $W_{\text{avail}} \leftarrow W$, batch counter $b \leftarrow 0$
Group completed jobs by distributor: $\{J_d\}_{d \in D}$
for each distributor d with job set J_d **do**
    if $W_{\text{avail}} \leq 0$ **then break**
    Remove dispatched jobs from J_d
    Sort J_d by heuristic score (descending), then by t_j^{comp} (stable sort)
    Initialize $\mathcal{G}_d \leftarrow \emptyset$, current group $G \leftarrow \emptyset$
    Set $t_{\text{start}} \leftarrow$ null
    for each $j \in J_d$ **do**
        if $t_{\text{start}} =$ null **then**
            $t_{\text{start}} \leftarrow t_j^{\text{comp}}$
        if $t_j^{\text{comp}} - t_{\text{start}} \leq \Delta t$ **then**
            Add j to G
        else
            if $G \neq \emptyset$ and $W_{\text{avail}} > 0$ **then**
                Add G to $\mathcal{G}_d$, decrement W_{avail}
            $G \leftarrow \{j\},\ t_{\text{start}} \leftarrow t_j^{\text{comp}}$
    if $G \neq \emptyset$ and $W_{\text{avail}} > 0$ **then**
        Add G to $\mathcal{G}_d$, decrement W_{avail}
    Assign batch IDs to $\mathcal{G}_d$: $\{(b + i, G_i)\}$, update $b \leftarrow b + |\mathcal{G}_d|$

Each batch is represented by a structured record:

$$\text{Batch} = (d, b, t_{\text{ready}}, p_{\text{batch}}, \mathcal{J}_b)$$

where d is the distributor ID, b is the batch ID, t_{ready} indicates the earliest dispatchable time, p_{batch} is the aggregated processing cost for dispatch preparation, and $\mathcal{J}_b$ denotes the set of jobs in the batch.

The dispatching agent is invoked whenever the meta-controller triggers a "dispatch" action. It receives the current environment state, filters out completed and undelivered jobs, applies the batching policy, and returns a structured dispatch plan in the form $\{\text{dispatch} : \{b_i : [j_1, j_2, \ldots]\}\}$. It ensures that no job is dispatched twice and that the number of active windows remains within the system constraint. This rule-based framework enables fast, explainable decision-making at the execution layer and complements the upper-level learning-based controller through modular coordination.

4.4 HDRL Training Procedure

The training of the hierarchical deep reinforcement learning (HDRL) framework is conducted in a simulated environment that replicates the dynamics of a flexible job shop with integrated scheduling and dispatching. The training procedure involves both high-level meta-policy learning and low-level sub-policy optimization, as outlined in Algorithm 3.

Algorithm 3 Hierarchical Scheduling and Dispatching Framework

Initialize Meta-Controller π_θ, Rule-DQN Q_ϕ, replay buffers, environment E
for episode $= 1$ to $E_{\max}$ **do**
  Reset environment: $s_0 \leftarrow E$.reset()
  for $t = 0$ to $T_{\max}$ **do**
    Observe high-level state s_t
    Compute high-level action probabilities $\pi_\theta(a_t \mid s_t)$
    Apply action mask and UCB bonus
    Select action $a_t \in \{\texttt{schedule}, \texttt{dispatch}, \texttt{wait}\}$
    if $a_t == \texttt{schedule}$ **then**
      while unscheduled operations remain **do**
        Extract local scheduling state x_t
        Apply Rule-DQN with rule filtering to select job-machine pair
        Execute assignment and update environment
        Store (x_t, a_t, r_t, x_{t+1}) in scheduling buffer
    else if $a_t == \texttt{dispatch}$ **then**
      Retrieve all completed jobs
      Group jobs by distributor ID
      for each distributor d **do**
        Apply time-window-based batching
        Compute priority score $\text{Score}(j, t) = -w_j(t)/p_j^2$
        Form batch groups and assign dispatch plans
    else
      Pass current time step
    Observe new state s_{t+1} and global reward r_t
    Store $(s_t, a_t, r_t, \log \pi_\theta(a_t), s_{t+1})$ in meta buffer
    if update condition met **then**
      Update Meta-Controller via policy gradient
      Update Rule-DQN via Q-learning
    if termination condition met **then**
      break

5 Computational Experiments

We evaluate the performance of the proposed HDRL framework through extensive computational experiments on benchmark instances of the IFJSSP-DP. The experiments are designed to assess the effectiveness of the hierarchical approach in comparison to traditional methods and to analyze the impact of various design choices.

5.1 Experimental Settings

In this section, we present the experimental settings used to evaluate the proposed hierarchical deep reinforcement learning (HDRL) framework. The experiments are conducted in a custom simulation environment that replicates the dynamics of a flexible job shop with integrated scheduling and dispatching. The objective is to assess the performance, generalizability, and adaptability of the proposed method under dynamic and large-scale production-distribution scenarios.

All experiments are implemented using Python 3.8 and PyTorch 1.12. The simulation environment emulates a shop floor with $M = 6$ heterogeneous machines and $D = 3$ distributors. Jobs arrive stochastically following a Poisson process with arrival rate $\lambda = 0.5$, each comprising 3 to 5 sequential operations. Operation processing times are uniformly drawn from $\mathcal{U}(5, 15)$, and machine assignment sets are randomly sampled per operation. The due date of each job j is

calculated using:

$$d_j = a_j + \delta_f \cdot \sum_{k=1}^{n_j} p_{jk} \tag{21}$$

where a_j is the job's arrival time, p_{jk} is the processing time of the k-th operation, and $\delta_f \in \{1.2, 1.5, 2.0\}$ is the delay factor used to simulate varying urgency levels. Delivery requirements for each distributor are defined via multiple time windows, with associated coverage ratios and penalty weights to simulate mass personalization demands.

For training, we adopt a small-scale configuration consisting of a single scenario with 190 dynamically arriving jobs. In contrast, the test dataset includes 30 randomly generated instances, each with 990 dynamic jobs and an identical statistical distribution. While the model is trained on a single delay factor $\delta_f = 1.5$, all three delay factors are employed during testing to evaluate generalization under variable urgency conditions.

The HDRL agent comprises a meta-controller and two specialized sub-agents. The meta-controller receives a 9-dimensional high-level system state and is modeled as a three-layer feedforward network with hidden size 32, GeLU activations, dropout regularization, and LayerNorm for stability. An Upper Confidence Bound (UCB) mechanism is used to balance exploration and exploitation. The scheduling agent is implemented as a Rule-DQN hybrid with eight heuristic policies and a two-layer MLP for Q-value approximation, trained via experience replay and soft target updates. The dispatching module follows a heuristic policy with reward shaping and constraint-driven penalties for tardiness and delivery coverage gaps.

Training is conducted over 800 episodes, each simulating a complete scheduling-dispatching horizon of $T = 200$ time steps. The learning rate is set to 10^{-3} for the meta-controller and 10^{-4} for the scheduling agent. An ϵ-greedy policy is adopted for the low-level agent with decaying exploration rate. A discount factor of $\gamma = 0.99$ is used throughout, reflecting the long-term nature of decision impact in dynamic environments. All experiments are performed on a desktop workstation equipped with an Intel i7-11700KF CPU, 32GB RAM, and an NVIDIA RTX 3070 GPU.

This experimental configuration enables us to rigorously examine the learned policy's robustness, scalability, and adaptability under different job complexities, system pressures, and delivery constraints.

Table 2: Parameters of hierarchical scheduling and dispatching scenarios.

Parameter	**Value**
Number of machines (m)	{10, 20, 30}
Number of initial jobs (n_{init})	20
Number of dynamically arrived jobs (n_{add})	{30, 50, 70}
Mean inter-arrival time (E_{arr})	{20, 40, 60}
Number of operations per job (n_i)	U[3, 6]
Operation processing time (t_{ij})	U[10, 50]
Job type (u_i)	B(1, 0)
Expiration time urgency factor ($ETUF_i$)	{1.5 ($u_i = 1$), 1 ($u_i = 0$)}
Due date of job j (δ_j)	$C_j^{\text{est}} + \Delta_j$, where $\Delta_j \sim \mathcal{U}(20, 40)$
Minimum batch size ($b_{\min}$)	{2, 3}
Maximum batch waiting time ($T_{\text{wait}}^{\max}$)	{20, 30}

Table 3: Reward function parameter configurations.

Parameter	Value
Scheduling reward coefficient (α_1, utilization)	2.0
Scheduling reward coefficient (α_2, progress rate)	2.0
Scheduling penalty coefficient (α_3, load imbalance)	1.5
Dispatching base reward (β_1)	2.0
Dispatching in-progress bonus (β_2)	0.5
Dispatching completion bonus (β_3)	1.0
Tardiness penalty ($r_j^{\text{tardiness}}$)	$-1 \cdot \max(0, d_j - \delta_j)$
Delivery shortfall penalty weight (w_r)	1.0
Operation completion reward (shaping)	+0.2
Job completion reward (shaping)	+0.5
Batch dispatch reward (shaping)	+1.0
Reward clipping range	$[-10, +10]$
Reward normalization method	Linear min-max scaling

5.2 Comparison with Heuristic Methods

To provide a comprehensive evaluation of the proposed hierarchical deep reinforcement learning (HDRL) framework, we compare it against four representative heuristic algorithms that are widely used in production scheduling and dispatching due to their interpretability, computational efficiency, and robustness in practice. These include the *Priority Rule Algorithm* (PR), which applies predefined dispatching rules such as earliest due date (EDD), shortest processing time (SPT), or minimum slack time to sequence jobs; the *Greedy Construction Algorithm* (GC), which incrementally builds a schedule by selecting job–machine assignments that yield the most immediate improvement; the *Local Search Algorithm* (LS), which iteratively refines an initial feasible schedule through neighborhood exploration such as job swaps and machine reassignments until no further improvement is found; and the *Genetic Algorithm* (GA), a population-based metaheuristic that evolves candidate schedules using selection, crossover, and mutation, guided by a multi-criteria fitness function. These methods represent a spectrum from simple constructive heuristics to advanced metaheuristics, offering a diverse baseline set for performance comparison.

Table 4: Total tardiness comparison of heuristic baselines and the proposed HDRL method.

Instance ID	#Jobs	#Machines	#Distributors	EDD	SPT	LPT	CR	Local Search	GA	Proposed
Small-01	8	4	2	149.0	152.5	160.3	148.7	133.2	128.7	120.4
Small-02	10	4	2	158.4	150.2	162.1	155.0	140.8	132.5	118.9
Small-03	12	4	2	162.0	159.3	165.7	160.4	138.9	130.1	116.7
Medium-01	40	8	3	480.3	470.2	498.6	465.7	422.1	408.9	390.5
Medium-02	60	8	3	690.7	672.5	710.3	675.4	633.0	610.4	588.2
Large-01	100	12	5	1150.6	1132.4	1180.9	1128.3	1075.2	1030.7	1005.9
Large-02	150	15	5	1720.8	1698.1	1754.6	1702.0	1620.5	1584.3	1530.4

Figure 1: Comparison of heuristic baselines and the proposed HDRL method: (a) total tardiness; (b) service-level metrics including on-time rate and request coverage.

As shown in Table 4, metaheuristic methods such as LS and GA generally outperform simple constructive heuristics like PR and GC in terms of reducing total tardiness. The GA achieves the best performance among the heuristic baselines, effectively balancing job completion timeliness and delivery coverage. Figure 1 further illustrates that the proposed HDRL framework consistently achieves superior results across both tardiness minimization and service-level metrics.

5.3 Comparison with single-level DRL Methods

We further compare our approach with two widely used flat deep reinforcement learning algorithms: Deep Q-Network (DQN) and Proximal Policy Optimization (PPO). Both models operate over the joint action space of scheduling and dispatching decisions, using a flattened state representation that lacks explicit temporal abstraction. DQN is

a value-based method that updates Q-values via temporal difference learning, employing ϵ-greedy exploration to balance exploitation and exploration. PPO, on the other hand, is a policy gradient method that uses a clipped surrogate objective to ensure stable training, enhanced by entropy regularization. Both methods are trained with the same reward formulation as our HDRL model to isolate the impact of structural differences in decision-making.

5.4 Comparison with Hierarchical DRL Methods

To benchmark the benefit of hierarchical design, we include two state-of-the-art HDRL baselines: Option-Critic (OC) and FeUdal Networks (FuN). The Option-Critic architecture jointly learns intra-option policies and termination functions, enabling multi-step temporal abstraction. FeUdal Networks employ a two-level structure in which a high-level manager issues subgoals, and a low-level worker acts accordingly, thereby separating strategic intent from local action execution. These models provide a strong reference for evaluating the modularity and adaptability of our proposed hierarchy.

Our proposed method consists of a high-level meta-controller that selects among scheduling, dispatching, and waiting actions using masked policy gradient optimization enhanced by an Upper Confidence Bound (UCB) exploration mechanism. The meta-controller is supported by a low-level Rule-DQN scheduling agent that integrates heuristic dispatching rules with deep Q-value estimation, and a dispatching module that uses reward-guided heuristics with explicit tardiness penalties and delivery constraint tracking. This multi-level coordination enables the system to address both local efficiency and global delivery objectives.

All models are evaluated on the same set of randomly generated job shop environments characterized by dynamic job arrivals, stochastic processing times, and time-sensitive delivery requirements. Each method is trained for 800 episodes and tested over 100 unseen scenarios. Evaluation metrics include average job tardiness, job completion rate, cumulative dispatch delay penalties, and per-step decision latency. These metrics capture both operational performance and real-time responsiveness. Quantitative results and performance comparisons are presented in Table **??**, with further analysis provided in Section **??**.

5.5 Ablation Study

To assess the contribution of key components in the HDRL framework, we conduct an ablation study by systematically removing or modifying specific elements and observing the impact on performance. The following variants are evaluated:

5.6 Results and Discussion

The results of the experiments demonstrate that the HDRL framework outperforms baseline methods in terms of solution quality and adaptability to changing conditions. The hierarchical structure allows for more efficient exploration of the solution space and better handling of complex dependencies between scheduling and dispatching tasks.

6 Conclusion

This paper presents a novel hierarchical reinforcement learning framework for solving the Integrated Flexible Job Shop Scheduling and Dispatching Problem (IFJSSP-DP). By decomposing the problem into high-level coordination and low-level scheduling-dispatching tasks, we leverage the strengths of both rule-based heuristics and deep learning to achieve efficient and adaptive decision-making in complex production environments. The proposed framework incorporates theoretical insights from stochastic scheduling literature to inform reward design and action prioritization, enhancing the learning process and system performance. The results demonstrate the effectiveness of the HDRL approach in managing the intricate interactions between scheduling and dispatching, leading to improved overall system performance and adaptability in dynamic environments.